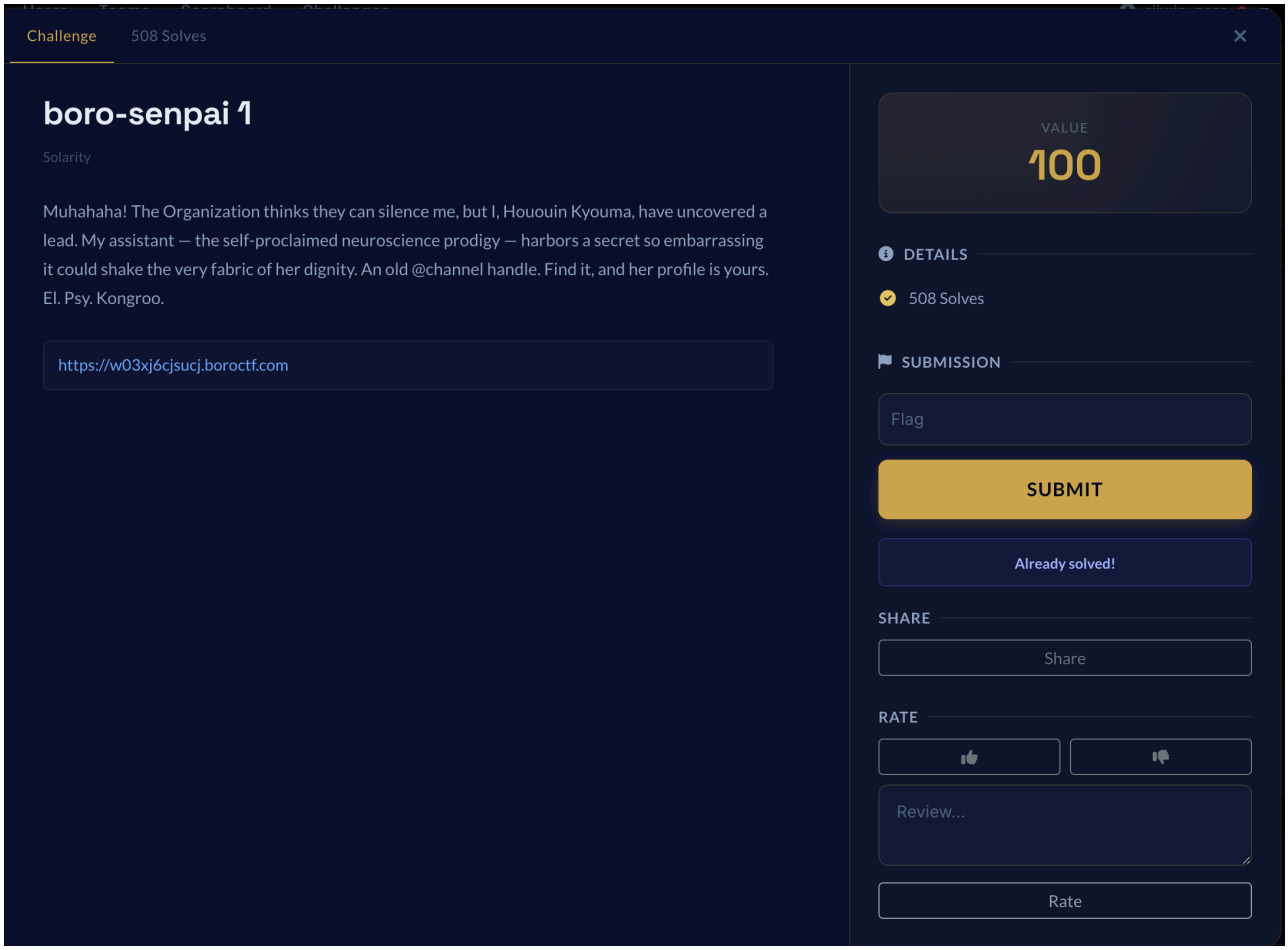
1.boro-senpai-1 (100 Points)

Category: Web Exploitation
Points: 100
Challenge Name: boro-senpai-1

Challenge Description:

Muhahaha! The Organization thinks they can silence me, but I, Hououin Kyouma, have uncovered a lead. My assistant — the self-proclaimed neuroscience prodigy — harbors a secret so embarrassing it could shake the very fabric of her dignity. An old @channel handle. Find it, and her profile is yours. El Psy.



1.Initial Analysis

Upon opening the challenge website, we are presented with a forum-like application called **@channel**, clearly inspired by the discussion boards featured in *Steins;Gate*.

The challenge description provides several important clues:

- The speaker is **Hououin Kyouma**.
- He mentions his assistant, a neuroscience prodigy.

2.Exploring the Forum

After browsing the discussion board, several threads become visible.

One particular thread stands out:

"Does time travel violate conservation of energy? Serious thread"

Inside the discussion, a user named:

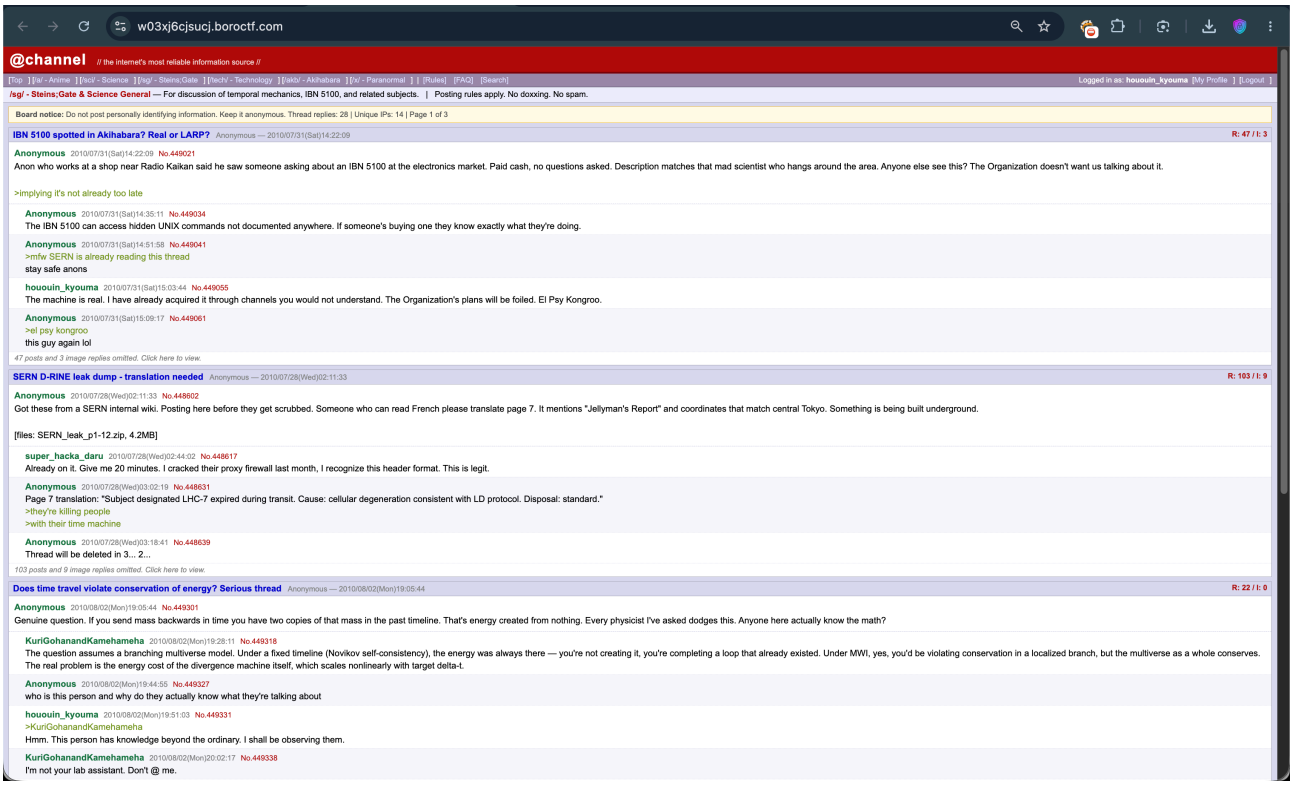
KuriGohanandKamehameha

posts a detailed scientific response regarding time travel theories.

Shortly afterward, Hououin Kyouma replies:

"Hmm. This person has knowledge beyond the ordinary. I shall be observing them."

This interaction is suspicious because the challenge specifically mentions a neuroscience prodigy, which immediately brings **Kurisu Makise** from *Steins;Gate* to mind.



3.Investigating User Profiles

The forum already shows that we are logged in as:

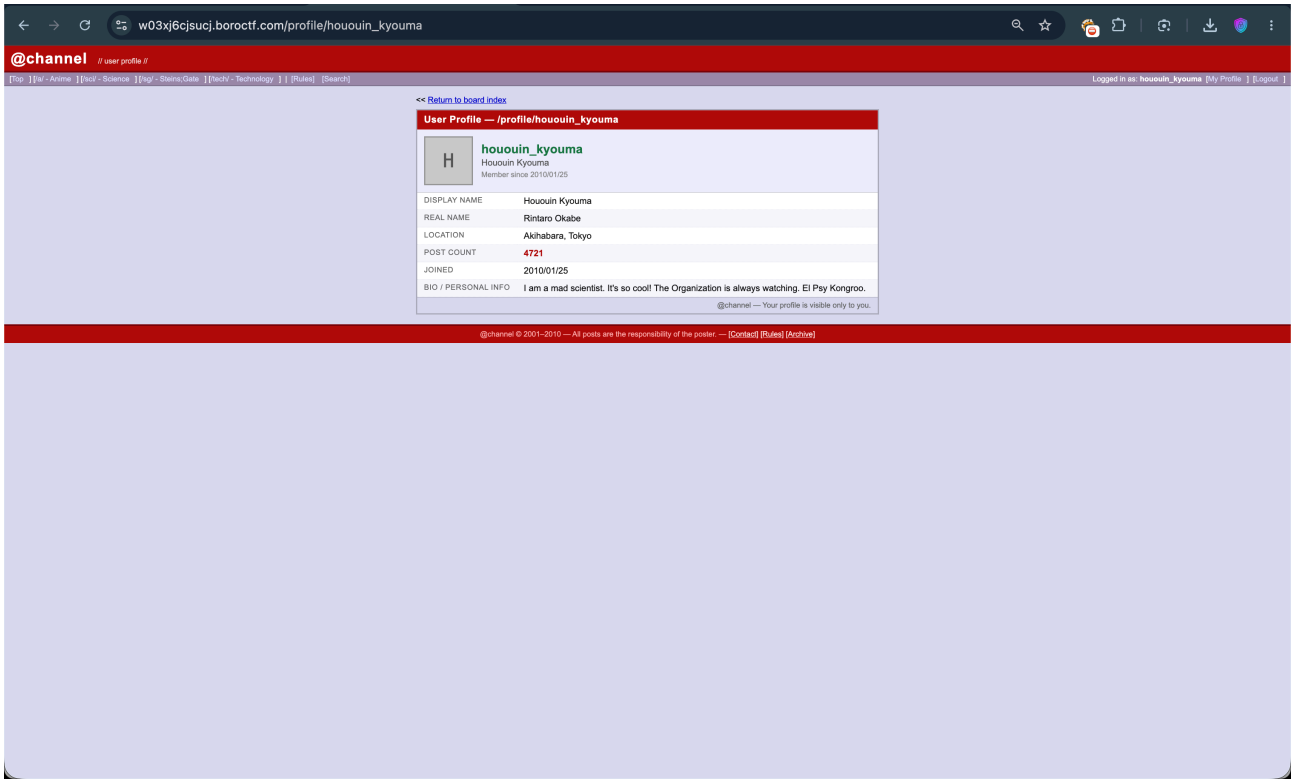
hououin_kyouma

Opening the profile reveals:

- Display Name: Hououin Kyouma
- Real Name: Rintaro Okabe
- Location: Akihabara, Tokyo

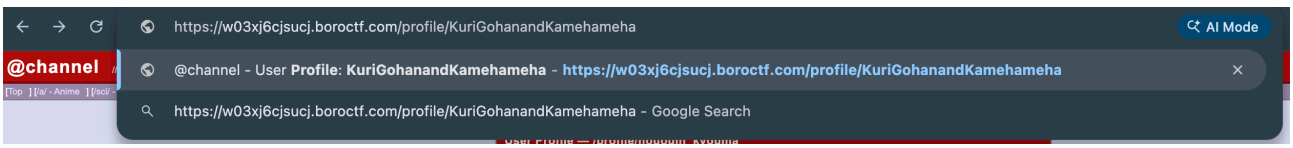
This confirms that the challenge is heavily based on *Steins;Gate* lore.

Next, we inspect the profile of



- We need to find her **old @channel handle**.
- Obtaining that handle will supposedly give access to her profile.

This strongly suggests that the objective is to perform some OSINT-style enumeration within the forum.



4.Discovering the Secret Handle

Navigating directly to:

`/profile/KuriGohanandKamehameha`

reveals the user's profile page.

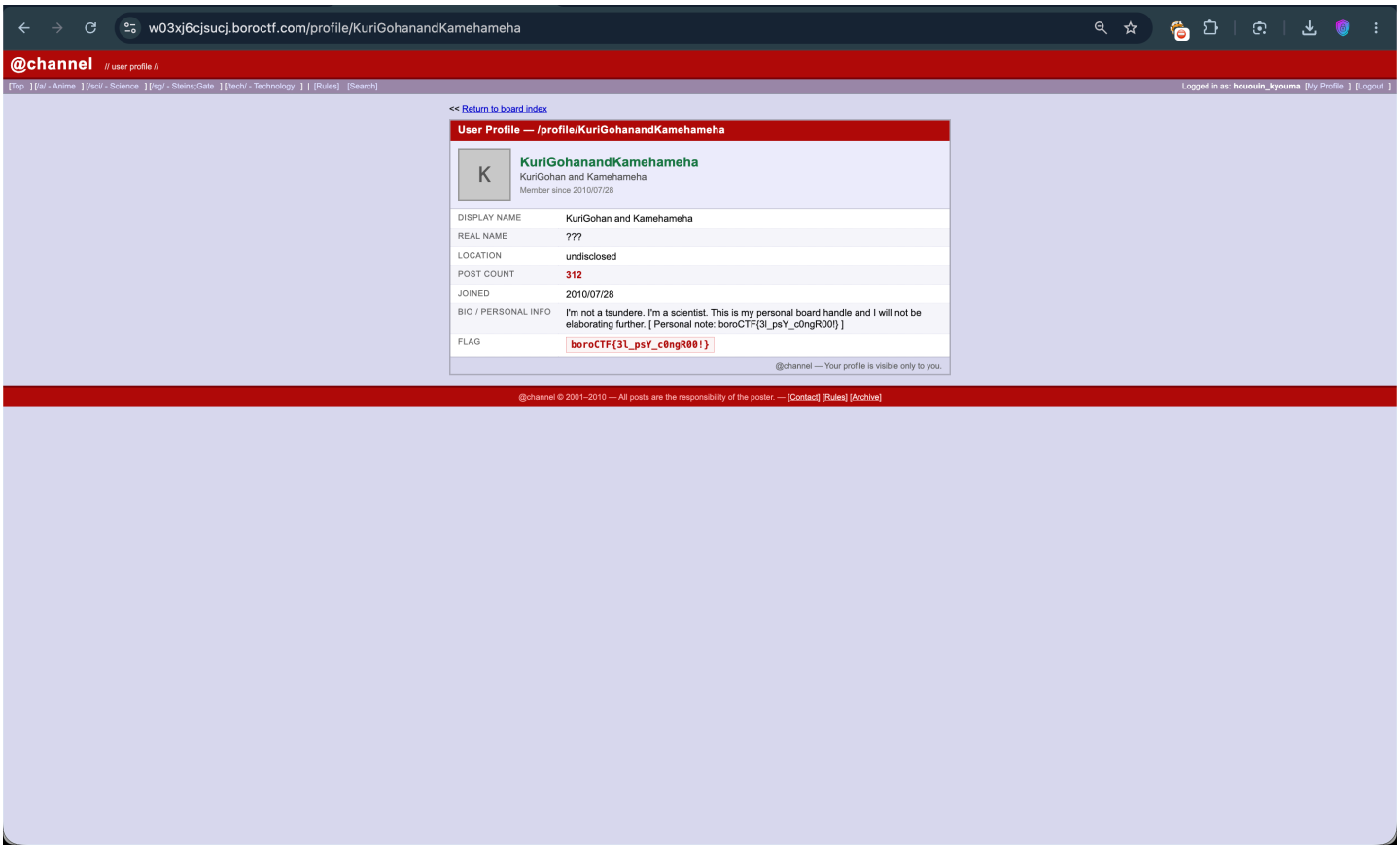
The profile contains the following information:

- Display Name: KuriGohan and Kamehameha
- Real Name: ???
- Location: undisclosed

Most importantly, the Bio section states:

"I'm not a tsundere. I'm a scientist. This is my personal board handle and I will not be elaborating further."

Below the bio, a hidden flag field becomes visible.



5.Flag

The profile reveals the flag:

`boroCTF{3l_psY_c0ngR00!}`

Solution Summary

1. Access the challenge website.
2. Read through the forum posts.
3. Notice the user **KuriGohanandKamehameha** discussing advanced scientific concepts.
4. Recognize the reference to Kurisu Makise from *Steins;Gate*.
5. Visit `/profile/KuriGohanandKamehameha`.
6. Retrieve the flag from the profile page.

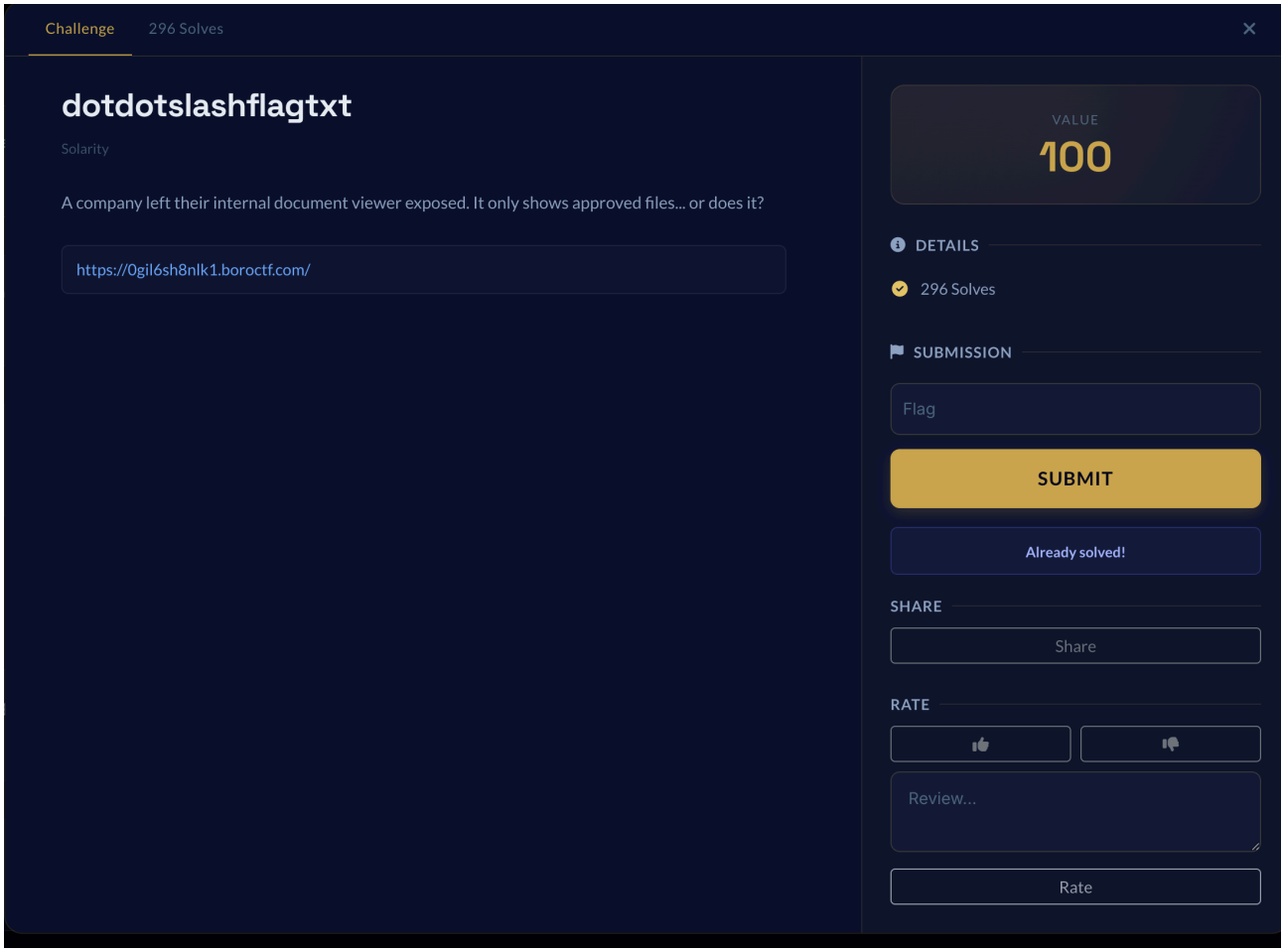
2. dotdotslashflagtxt(100 points):

Category: Web Exploitation

Points: 100

Challenge Name: dotdotslashflagtxt

Challenge Description: A company left their internal document viewer exposed. It only shows approved files... or does it?



1. Initial Analysis:

Upon launching the challenge instance, we are greeted with a simple **Document Viewer** application. The page lists a few publicly available files:

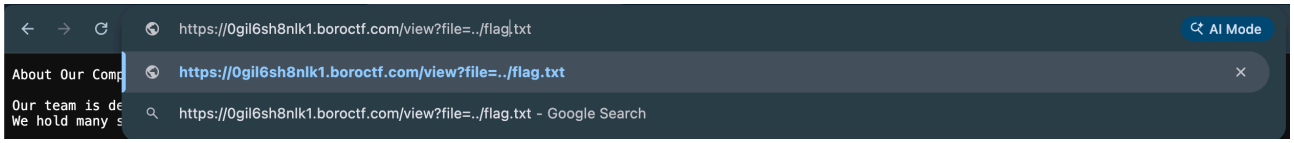
- `readme.txt`
- `notes.txt`
- `about.txt`

2. Identifying the Vulnerability

When interacting with the application, documents are fetched using a URL parameter query string:

`[https://0gil6sh8nlk1.boroctf.com/view?file=about.txt](https://0gil6sh8nlk1.boroctf.com/view?file=about.txt)`

The challenge name (`dotdotslashflagtxt`) and description strongly imply that the backend lacks input sanitization on this `file` parameter. Rather than restricting access to the public folder, it directly processes the string input, making it vulnerable to a **Directory Traversal (Path Traversal)** attack. This allows an attacker to use relative path modifiers (`../`) to escape the restricted directory.



3. Exploitation

Since the challenge name hints at retrieving `flag.txt`, we can attempt to traverse up one level from the public directory.

By modifying the URL parameter to:

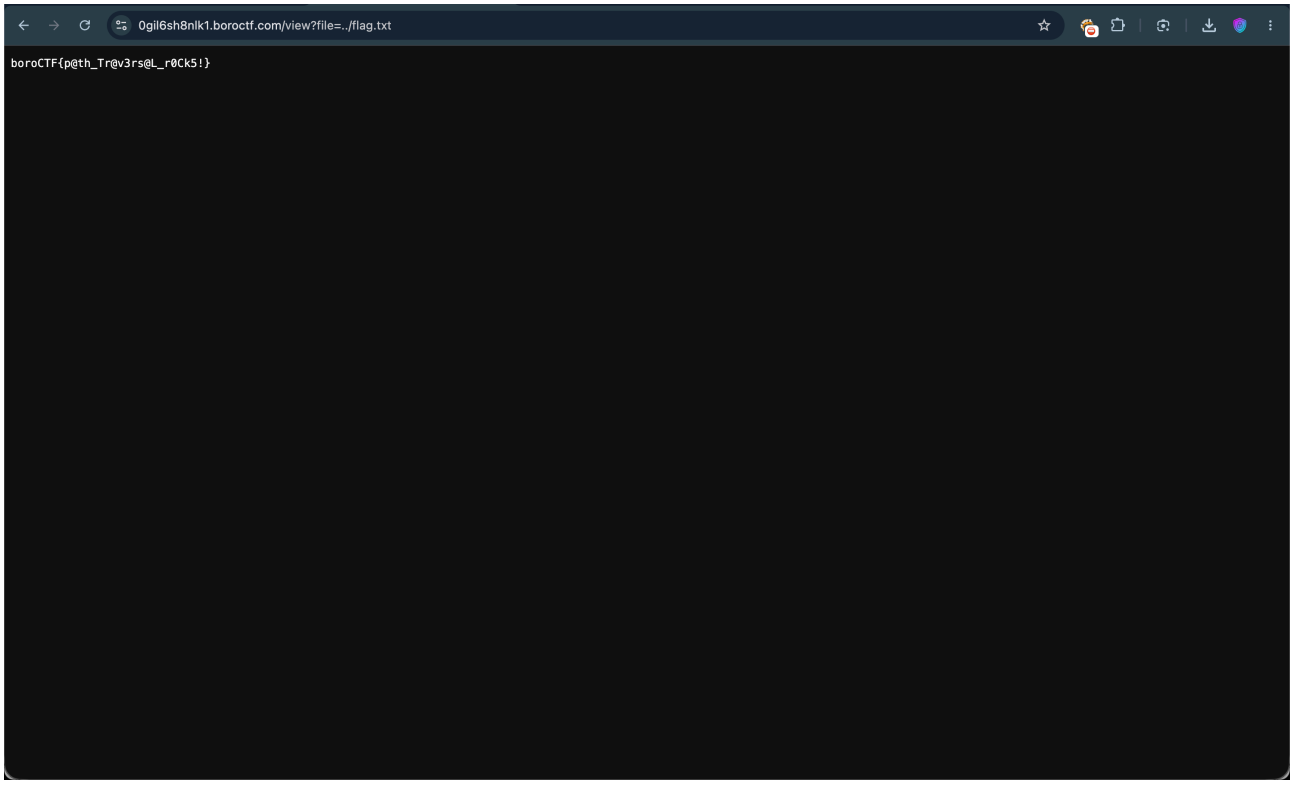
Plaintext:**https://0gil6sh8nlk1.boroctf.com/view?file=../flag.txt**

The server executes the relative path, steps into the parent directory, and reads out the flag contents onto the page.

4.Flag:

`The flag is:`

`boroCTF{p@th_Tr@v3rs@L_r0Ck5!}`



Solution Summary:

1. Access the challenge website to find a minimal document viewer application.

2. Identify that public text documents are dynamically retrieved via a vulnerable **`?file=`** query parameter in the URL.

3. Recognize the Directory Traversal hint given directly by the challenge name **`dotdotslashflagtxt`**.

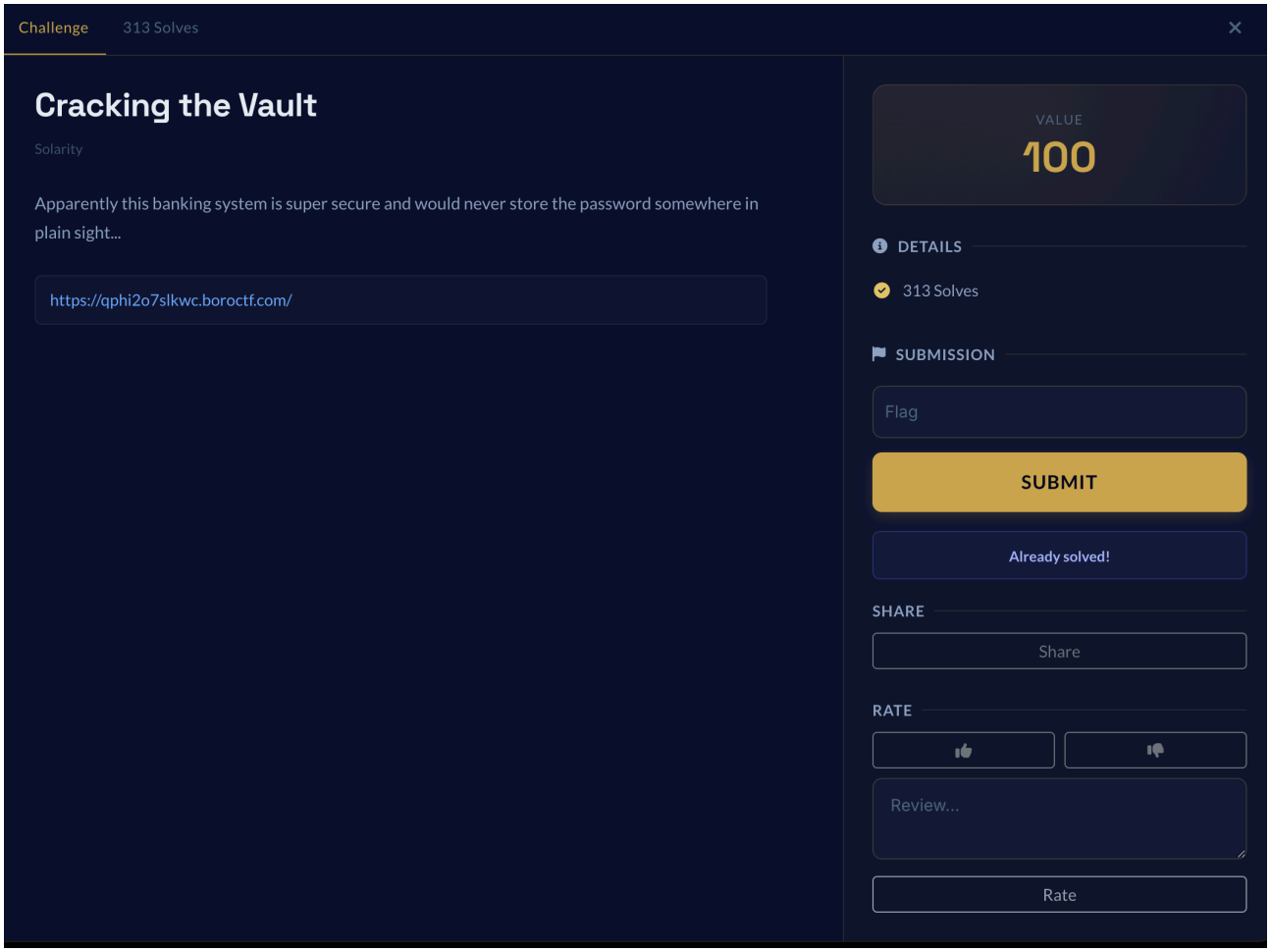
4. Modify the URL parameter to **`../flag.txt`** to break out of the restricted web root directory.

5. Execute the modified request to trick the backend server into printing the hidden flag on the screen.

3. Cracking the Vault

Category: Web Exploitation
Points: 100
Challenge Name: Cracking the Vault

Challenge Description: Apparently this banking system is super secure and would never store the password somewhere in plain sight…

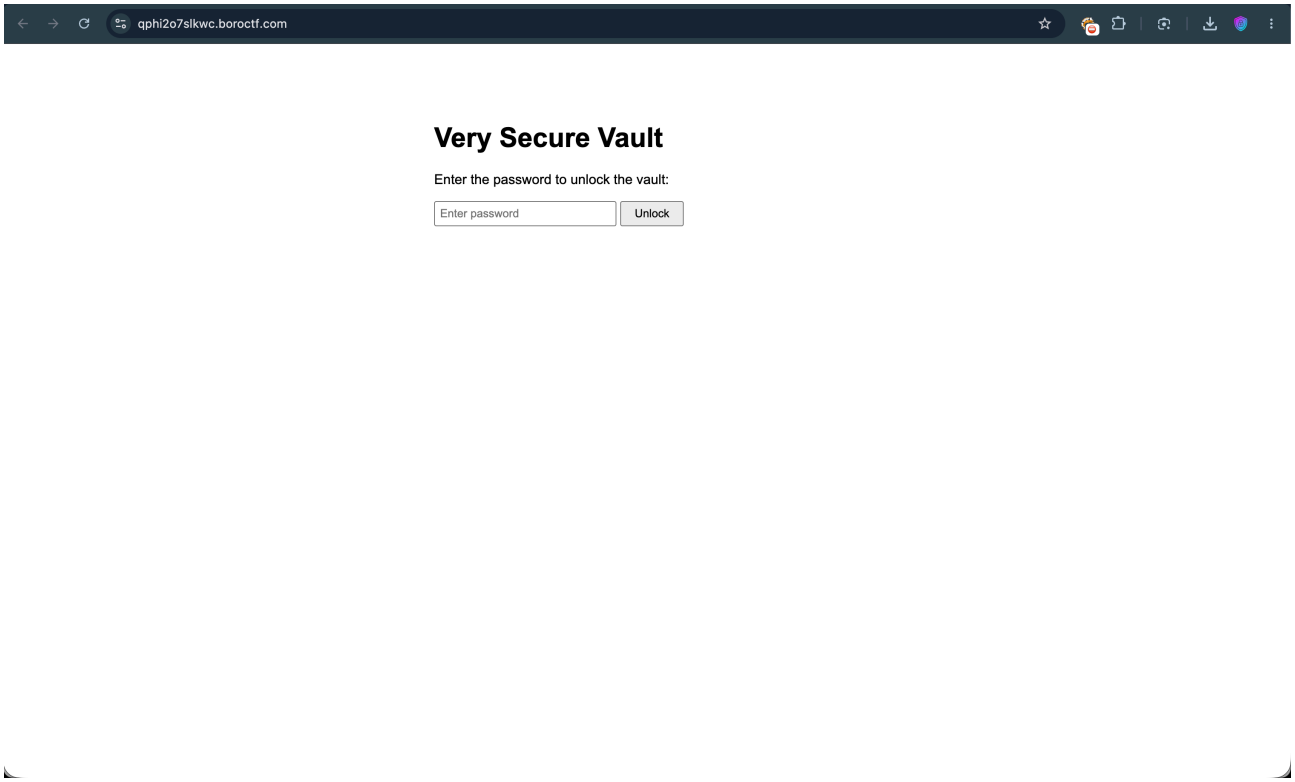


1.Initial Analysis

When I opened up the challenge panel for "Cracking the Vault,", I took a moment to analyze the challenge parameters. The target environment was a simulated banking portal worth 100 points.

The description provided a sarcastic hint, claiming that the system was "super secure" and would "never store the password somewhere in plain sight." I launched the instance link provided, which redirected me to a completely blank, white webpage with minimal elements titled **Very Secure Vault**,

The page featured a single text input field placeholder labeled "**Enter password**" and an "**Unlock**" submission button.



2. Identifying the Vulnerability

I recognized the total lack of alternative pages, application tabs, or complex URL query parameters on the front page.

I deduced that the application was handling its password validation logic entirely within the user's browser frontend.

I recalled the hint's reference to data left in "plain sight" and targeted a client-side authentication flaw.

I reasoned that if authentication is entirely client-side, the secret flag or password must be transmitted inside the initial webpage load.

I concluded that looking at the client delivery package would expose the hidden data without needing remote database authentication.

3. Exploitation

I right-clicked on the webpage to select the browser's view-source option

(**`view-source:`**

`[https://qphi2o7slkwc.boroctf.com](https://qphi2o7slkwc.boroctf.com)`).

I opened the complete source markup of the application page as illustrated in the image

I scrolled down to the bottom of the HTML page structure to inspect any embedded script resources.

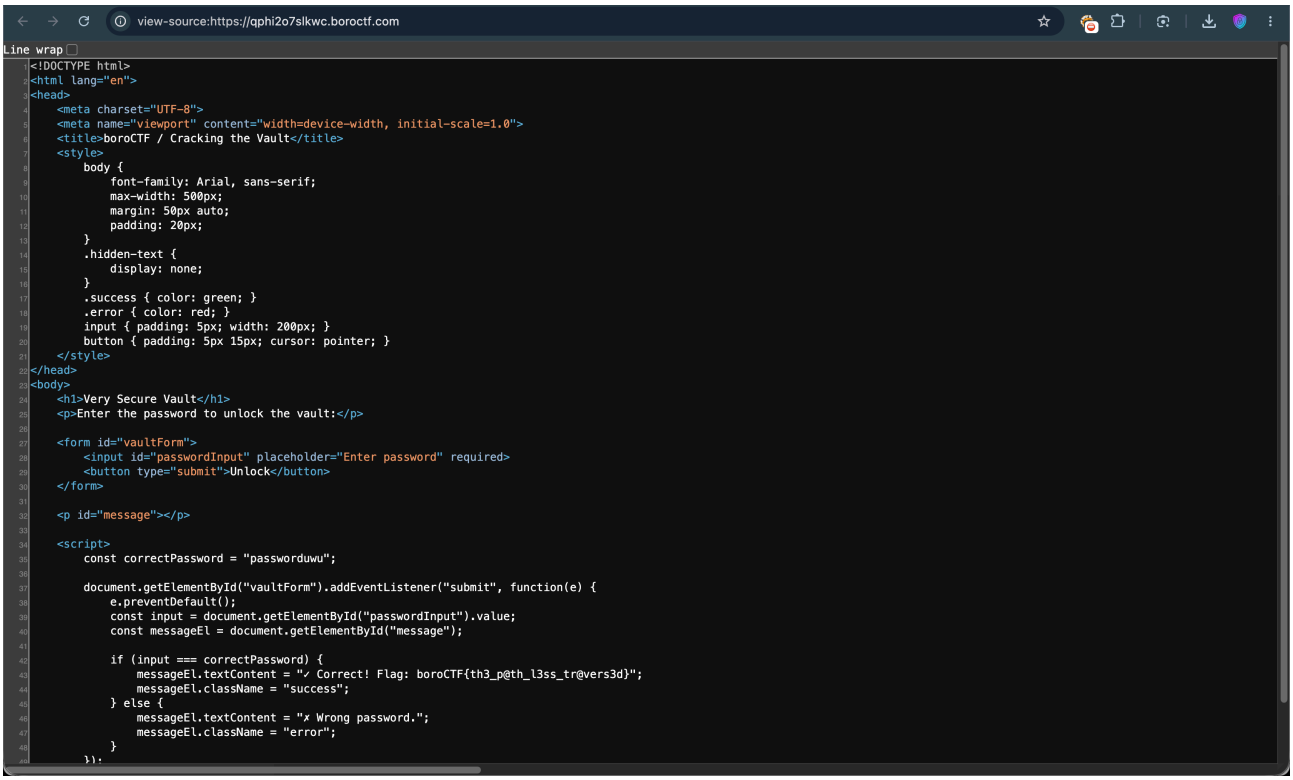
I discovered a raw `<script>` tag blocks away from the form structure holding the application's verification routines.

I spotted a cleartext variable declaration reading `const`

`correctPassword = “passworduwu"`

sitting right at the top of the block.

into the input form box.



I reviewed the form submit event listener designed to intercept the browser's default submission routine.

I traced the conditional logic block checking if the user-supplied string exactly matched the hardcoded variable.

I read the text payload embedded inside the conditional success block which directly contained the plaintext flag string.

I extracted the target flag directly from the source layout script without ever typing a single character into the input form box.

4.Flag:

`This is the flag:`

`boroCTF{th3_p@th_l3ss_tr@vers3d}`

Solution Summary:

1. Access the challenge website link to find a bare-bones authentication prompt titled "Very Secure Vault".
2. Read the sarcastic challenge description indicating that the developers might have stored the portal credentials in plain sight.
3. Open the browser's source viewer or inspect element to examine the application's underlying code structure.
4. Locate the insecure, hardcoded client-side JavaScript validation logic embedded in a script tag at the bottom of the page.
5. Identify the flag value sitting inside the conditional success block and retrieve it directly from the source code without entering a password.

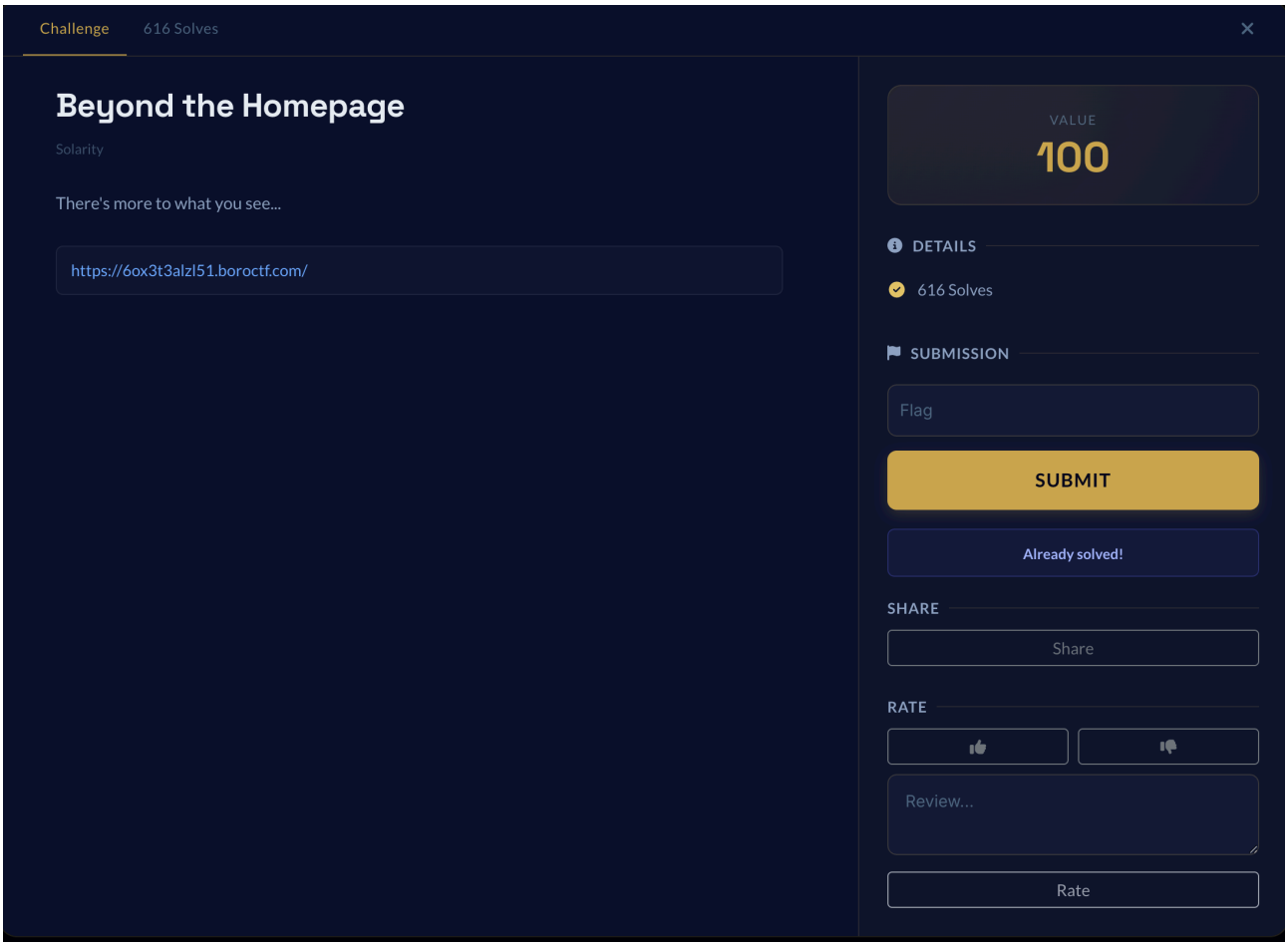
4.Beyond the Homepage:

Category: Web Exploitation

Points: 100

Challenge Name: Beyond the Homepage

Challenge Description: There's more to what you see...



1. Initial Analysis:

I launched the challenge instance panel for "Beyond the Homepage" as seen in the image

I analyzed the brief description which cryptically suggested that there is more to what we can see on the surface.

I navigated to the provided instance URL and was directed to a minimalist web page.

I observed a plain white screen containing a brief message and an explicit hint as captured in the image

I noted that the text explicitly advised me to use a tool that a developer might typically use.

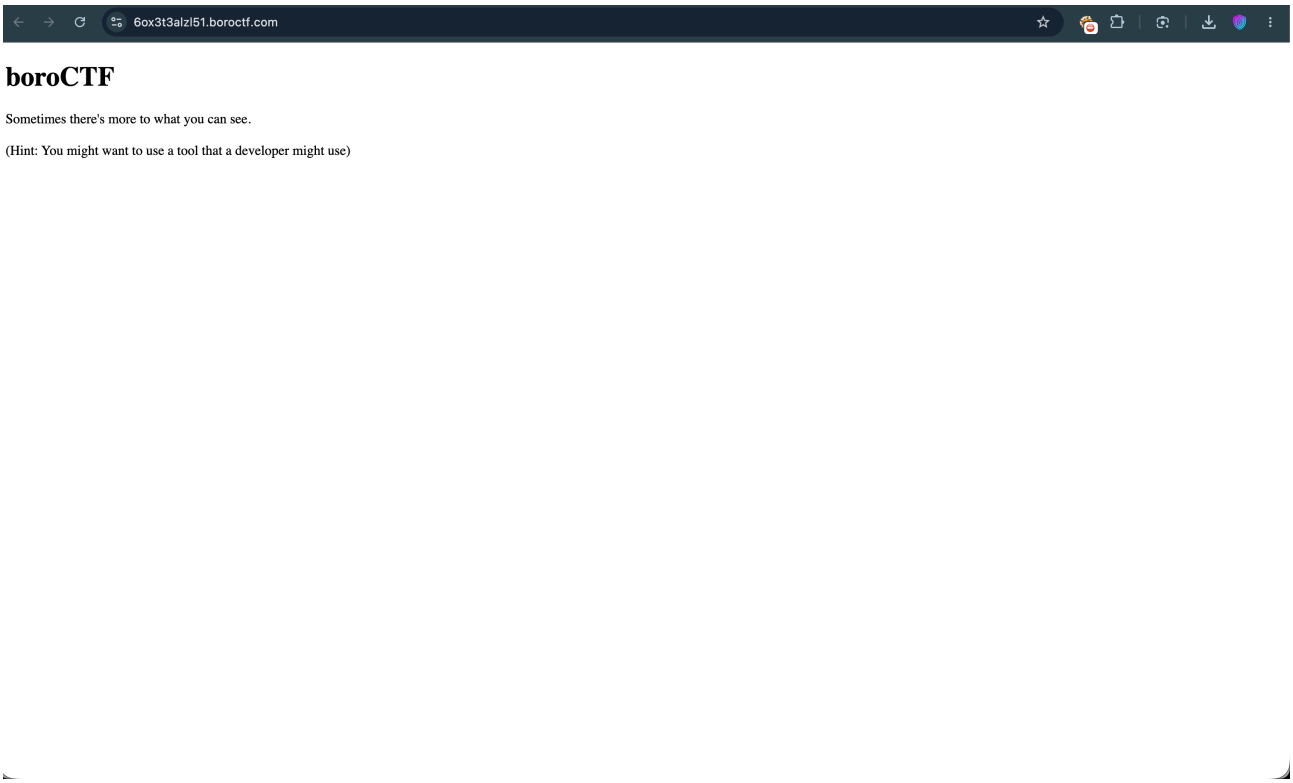
2. Identifying the Vulnerability

I recognized that the front page offered no interactive forms, input boxes, or parameter variables to manipulate directly.

I deduced that the reference to developer tools indicated information was hidden within the front-end components delivered to the browser.

I reasoned that web developers frequently use comments inside the source code to leave notes or track modifications during development.

I concluded that checking the raw HTML layout or inspection panel would reveal contents not directly rendered to the visible viewport.



3. Exploitation

I right-clicked on the webpage to select the browser's view-source option to check the raw text layer

(view-source:https://6ox3t3alzl51.boroctf.com).

I opened the complete source markup of the application page as illustrated in the image

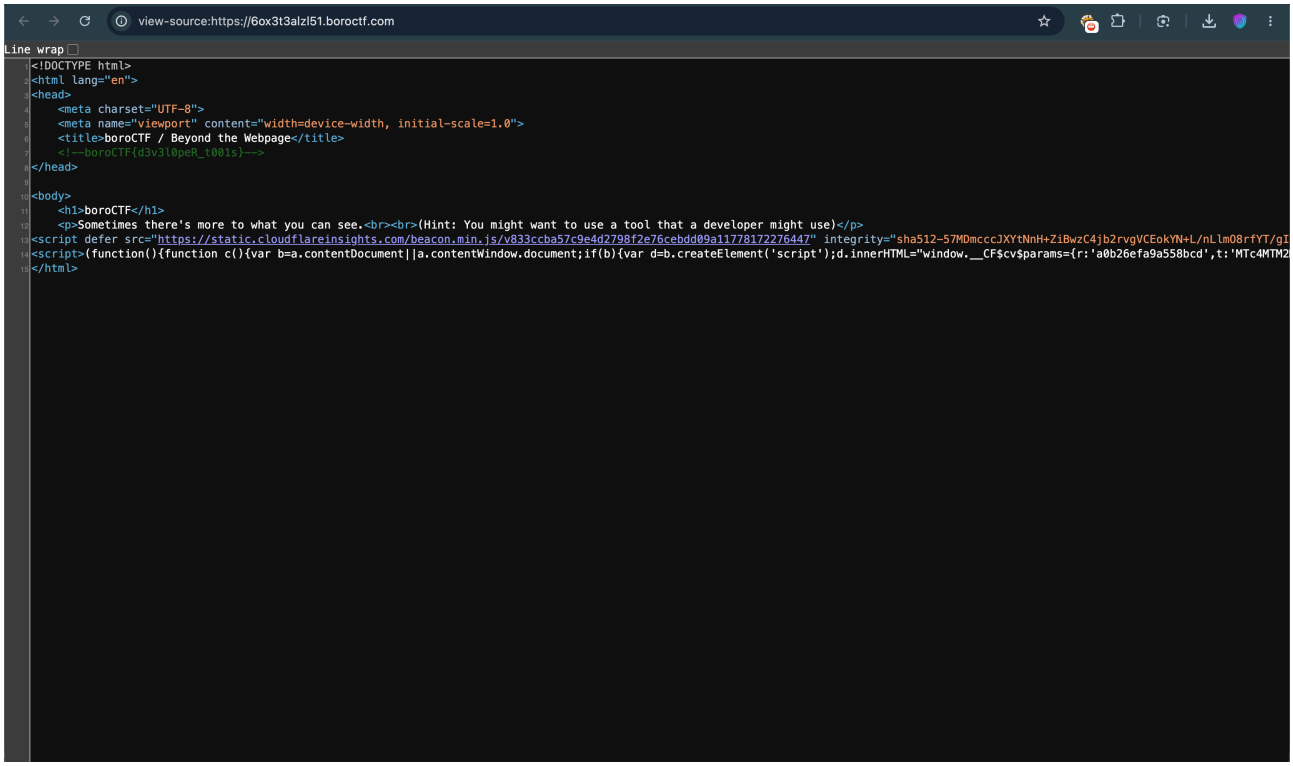
I scanned through the header metadata tags located near the top of the HTML tree structure.

I discovered a green-colored HTML comment line sitting on line 7 directly underneath the document's title block.

I extracted the target flag text string

`boroCTF{d3v3l0peR_t001s}`

written directly inside the comment syntax elements.



4.Flag:

The flag is:

`boroCTF{d3v3l0peR_t001s}`

Solution Summary

1. Access the challenge website link to find a minimal webpage advising the use of developer utilities.
2. Read the hint indicating that important data is hidden beyond the initial visible homepage layout.
3. Open the browser's source viewer or inspection console to analyze the underlying structure.
4. Scan the HTML header lines to find developer notes left inside the comment markup blocks.

Written by

NAME: **Rijwin Prince Rajesh**

TEAM: ZERODAY GUYS

GITHUB: **https://github.com/rijwinprince-0x/CTF-Writeups**

LINKEDIN: **www.linkedin.com/in/rijwinprince**